```python
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix, accuracy_score,
classification_report
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
!pip install -U scikit-learn
```

```
Requirement already satisfied: scikit-learn in
/usr/local/lib/python3.12/dist-packages (1.8.0)
Requirement already satisfied: numpy>=1.24.1 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.10.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.3.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
```

```python
def importdata():
    url = "https://archive.ics.uci.edu/static/public/12/data.csv"
    balance_data = pd.read_csv(url, header=0)
    print("Dataset Length:", len(balance_data))
    print("Dataset Shape:", balance_data.shape)
    print("Dataset Head:\n", balance_data.head())
    return balance_data

def splitdataset(balance_data):
    X = balance_data.iloc[:, 1:5].values
    Y = balance_data.iloc[:, 0].values
    X_train, X_test, y_train, y_test = train_test_split(X, Y,
test_size=0.3, random_state=100)
    return X, Y, X_train, X_test, y_train, y_test

def train_using_gini(X_train, y_train):
    clf_gini = DecisionTreeClassifier(criterion="gini",
random_state=100, max_depth=3, min_samples_leaf=5)
    clf_gini.fit(X_train, y_train)
    return clf_gini

def train_using_entropy(X_train, y_train):
    clf_entropy = DecisionTreeClassifier(criterion="entropy",
random_state=100, max_depth=3, min_samples_leaf=5)
    clf_entropy.fit(X_train, y_train)
    return clf_entropy

def prediction(X_test, clf_object):
    y_pred = clf_object.predict(X_test)
    print("Predicted Values:\n", y_pred)
    return y_pred
```

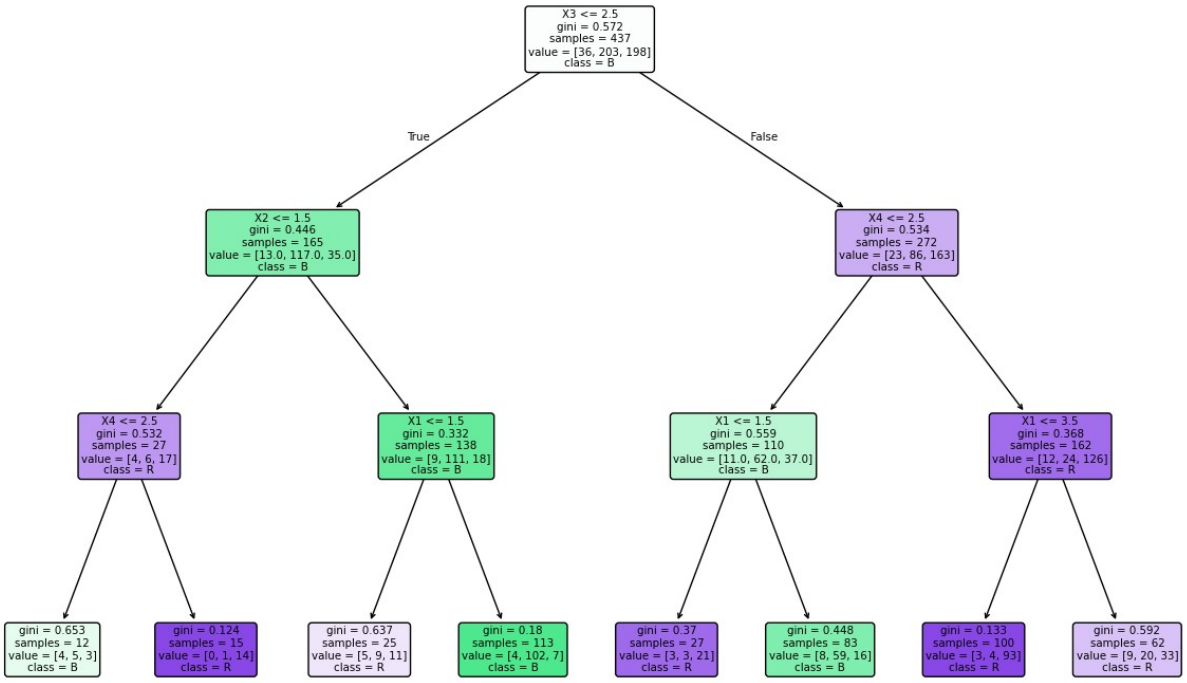
```python
def cal_accuracy(y_test, y_pred):
    print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("\nAccuracy:", accuracy_score(y_test, y_pred) * 100)
    print("\nClassification Report:\n", classification_report(y_test,
y_pred))

def plot_decision_tree(clf_object, feature_names, class_names):
    plt.figure(figsize=(15, 10))
    plot_tree(clf_object, filled=True, feature_names=feature_names,
class_names=class_names, rounded=True)
    plt.show()

if __name__ == "__main__":

    data = importdata()
    X, Y, X_train, X_test, y_train, y_test = splitdataset(data)


    print("\n----- Training Using Gini -----")
    clf_gini = train_using_gini(X_train, y_train)
    y_pred_gini = prediction(X_test, clf_gini)
    cal_accuracy(y_test, y_pred_gini)


    print("\n----- Training Using Entropy -----")
    clf_entropy = train_using_entropy(X_train, y_train)
    y_pred_entropy = prediction(X_test, clf_entropy)
    cal_accuracy(y_test, y_pred_entropy)
```

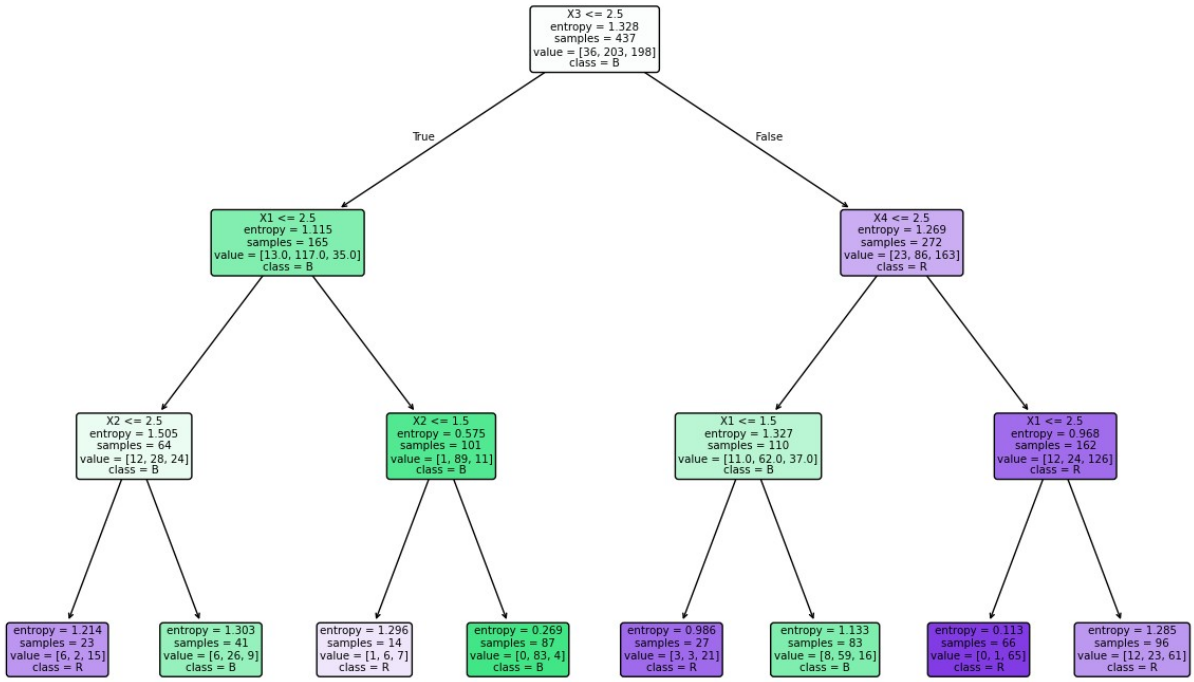
```
Dataset Length: 625
Dataset Shape: (625, 5)
Dataset Head:
   class  left-weight  left-distance  right-weight  right-distance
0      B            1              1             1               1
1      R            1              1             1               2
2      R            1              1             1               3
3      R            1              1             1               4
4      R            1              1             1               5

----- Training Using Gini -----
Predicted Values:
 ['R' 'L' 'R' 'R' 'R' 'L' 'R' 'L' 'L' 'L' 'R' 'L' 'L' 'L' 'R' 'L' 'R'
'L'
 'L' 'R' 'L' 'R' 'L' 'L' 'R' 'L' 'L' 'L' 'R' 'L' 'L' 'L' 'R' 'L' 'L'
'L'
 'L' 'R' 'L' 'L' 'R' 'L' 'R' 'L' 'R' 'R' 'L' 'L' 'R' 'L' 'R' 'R' 'L'
'R'
 'R' 'L' 'R' 'R' 'L' 'L' 'R' 'R' 'L' 'L' 'L' 'L' 'L' 'R' 'R' 'L' 'L'
'R'
 'R' 'L' 'R' 'L' 'R' 'R' 'R' 'L' 'R' 'L' 'L' 'L' 'L' 'R' 'R' 'L' 'R'
'L'
```

```
 'R' 'R' 'L' 'L' 'L' 'R' 'R' 'L' 'L' 'L' 'R' 'L' 'R' 'R' 'R' 'R' 'R'
'R'
 'R' 'L' 'R' 'L' 'R' 'R' 'L' 'R' 'R' 'R' 'R' 'R' 'L' 'R' 'L' 'L' 'L'
'L'
 'L' 'L' 'L' 'R' 'R' 'R' 'R' 'L' 'R' 'R' 'R' 'L' 'L' 'R' 'L' 'R' 'L'
'R'
 'L' 'L' 'R' 'L' 'L' 'R' 'L' 'R' 'L' 'R' 'R' 'R' 'L' 'R' 'R' 'R' 'R'
'R'
 'L' 'L' 'R' 'R' 'R' 'R' 'L' 'R' 'R' 'R' 'L' 'R' 'L' 'L' 'L' 'L' 'R'
'R'
 'L' 'R' 'R' 'L' 'L' 'R' 'R' 'R']

Confusion Matrix:
 [[ 0  6  7]
 [ 0 67 18]
 [ 0 19 71]]

Accuracy: 73.40425531914893

Classification Report:
               precision    recall  f1-score   support

           B       0.00      0.00      0.00        13
           L       0.73      0.79      0.76        85
           R       0.74      0.79      0.76        90

    accuracy                           0.73       188
   macro avg       0.49      0.53      0.51       188
weighted avg       0.68      0.73      0.71       188


----- Training Using Entropy -----
Predicted Values:
 ['R' 'L' 'R' 'L' 'R' 'L' 'R' 'L' 'R' 'R' 'R' 'R' 'L' 'L' 'R' 'L' 'R'
'L'
 'L' 'R' 'L' 'R' 'L' 'L' 'R' 'L' 'R' 'L' 'R' 'L' 'R' 'L' 'R' 'L' 'L'
'L'
 'L' 'L' 'R' 'L' 'R' 'L' 'R' 'L' 'R' 'R' 'L' 'L' 'R' 'L' 'L' 'R' 'L'
'L'
 'R' 'L' 'R' 'R' 'L' 'R' 'R' 'R' 'L' 'L' 'R' 'L' 'L' 'R' 'L' 'L' 'L'
'R'
 'R' 'L' 'R' 'L' 'R' 'R' 'R' 'L' 'R' 'L' 'L' 'L' 'L' 'R' 'R' 'L' 'R'
'L'
 'R' 'R' 'L' 'L' 'L' 'R' 'R' 'L' 'L' 'L' 'R' 'L' 'L' 'R' 'R' 'R' 'R'
'R'
 'R' 'L' 'R' 'L' 'R' 'R' 'L' 'R' 'R' 'L' 'R' 'R' 'L' 'R' 'R' 'R' 'L'
'L'
 'L' 'L' 'L' 'R' 'R' 'R' 'R' 'L' 'R' 'R' 'R' 'L' 'L' 'R' 'L' 'R' 'L'
'R'
 'L' 'R' 'R' 'L' 'L' 'R' 'L' 'R' 'R' 'R' 'R' 'R' 'L' 'R' 'R' 'R' 'R'
```

```
'R'
 'R' 'L' 'R' 'L' 'R' 'R' 'L' 'R' 'L' 'R' 'L' 'R' 'L' 'L' 'L' 'L' 'L'
'R'
 'R' 'R' 'L' 'L' 'L' 'R' 'R' 'R']

Confusion Matrix:
 [[ 0  6  7]
 [ 0 63 22]
 [ 0 20 70]]

Accuracy: 70.74468085106383

Classification Report:
              precision    recall  f1-score   support

           B       0.00      0.00      0.00        13
           L       0.71      0.74      0.72        85
           R       0.71      0.78      0.74        90

    accuracy                           0.71       188
   macro avg       0.47      0.51      0.49       188
weighted avg       0.66      0.71      0.68       188


/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1833: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
result.shape[0])
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatio
n.py:1833: UndefinedMetricWarning: Precision is ill-defined and being
set to 0.0 in labels with no predicted samples. Use `zero_division`
parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
result.shape[0])
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatio
n.py:1833: UndefinedMetricWarning: Precision is ill-defined and being
set to 0.0 in labels with no predicted samples. Use `zero_division`
parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
result.shape[0])
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatio
n.py:1833: UndefinedMetricWarning: Precision is ill-defined and being
set to 0.0 in labels with no predicted samples. Use `zero_division`
parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
result.shape[0])
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatio
n.py:1833: UndefinedMetricWarning: Precision is ill-defined and being
```

```
set to 0.0 in labels with no predicted samples. Use `zero_division`
parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
result.shape[0])
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatio
n.py:1833: UndefinedMetricWarning: Precision is ill-defined and being
set to 0.0 in labels with no predicted samples. Use `zero_division`
parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
result.shape[0])

plot_decision_tree(clf_gini, ['X1', 'X2', 'X3', 'X4'], ['L', 'B',
'R'])

plot_decision_tree(clf_entropy, ['X1', 'X2', 'X3', 'X4'], ['L', 'B',
'R'])
```




X3 <= 2.5
entropy = 1.328
samples = 437
value = [36, 203, 198]
class = B
True
False
X1 <= 2.5
entropy = 1.115
samples = 165
value = [13.0, 117.0, 35.0]
class = B
X4 <= 2.5
entropy = 1.269
samples = 272
value = [23, 86, 163]
class = R
X2 <= 2.5
entropy = 1.505
samples = 64
value = [12, 28, 24]
class = B
X2 <= 1.5
entropy = 0.575
samples = 101
value = [1, 89, 11]
class = B
X1 <= 1.5
entropy = 1.327
samples = 110
value = [11.0, 62.0, 37.0]
class = B
X1 <= 2.5
entropy = 0.968
samples = 162
value = [12, 24, 126]
class = R
entropy = 1.214
samples = 23
value = [6, 2, 15]
class = R
entropy = 1.303
samples = 41
value = [6, 26, 9]
class = B
entropy = 1.296
samples = 14
value = [1, 6, 7]
class = R
entropy = 0.269
samples = 87
value = [0, 83, 4]
class = B
entropy = 0.986
samples = 27
value = [3, 3, 21]
class = R
entropy = 1.133
samples = 83
value = [8, 59, 16]
class = B
entropy = 0.113
samples = 66
value = [0, 1, 65]
class = R
entropy = 1.285
samples = 96
value = [12, 23, 61]
class = R